

Searching Gapped Palindromes using Inverted Suffix Array

Shivika Gupta

Dept. of Computer Science
Ajay Kumar Garg Engineering
College
Ghaziabad, India
guptashivika05@yahoo.com

Rajesh Prasad

Dept. of Computer Science
Yobe State University
Damaturu, Nigeria
rajesh_ucer@yahoo.com

Sunita Yadav

Dept. of Computer Science
Ajay Kumar Garg Engineering
College
Ghaziabad, India
yadav.sunita104@gmail.com

Abstract— Palindrome pattern matching is a classical and well-studied problem in computer science. A palindrome is a string that reads the same forward and backward. Gapped palindrome is an interesting version of the palindrome which is defined as the one having a space between left and right palindromic arms of the string. In this paper, we develop efficient algorithms to detect two different classes of gapped palindromes: long armed and length constrained in a biological sequences by using inverted suffix array. The algorithms perform the computation in $O(n)$ time. Also, we determine palindromic weights (number and size of gapped palindromes) in the input biological string.

Keywords— Palindromes, Gapped Palindromes, Long Armed, Length Constrained, Inverted Suffix Array, Longest Common Prefix and Palindromic Weight.

I. INTRODUCTION

Palindrome pattern matching problem is a classical and well-studied problem in computer science with a lot of variations arising out of different practical scenarios [2]. A palindrome [14, 17, 18, 19, 20] is a string that reads the same forward and backward. In this paper, we study gapped palindromes: sub words of the form vuv^T . It is a gapped palindrome for some u, v , where v^T is reverse of v . Occurrences of v, v^T and u are called respectively left, right arm of the palindrome and spacer (gap) between them. They have two natural classes [1]: the first class is long-armed palindromes, it verifies the condition $|u| \leq |v|$, and i.e. length of the palindrome arm is not less than the length of the spacer. The second class is length-constrained palindromes and is specified by lower and upper length bounds on the spacer length such that $\text{MinGap} \leq |u| \leq \text{MaxGap}$, and a lower bound on the arm length such that $\text{MinArm} \leq |v|$, where MinGap , MaxGap , MinArm are pre-defined constants.

String and sequence algorithms related to palindromes have attracted stringology [8] researchers since a long time [1, 3, 7]. Gapped Palindromes also seems to be a new interesting addition to the rich list of problems related to palindromes. The palindromic structures found in biological sequences play a role in regulation of gene activity or other processes in cells [1, 7, 9, 16]. The problem of computing palindromes and its related variants has received some attention in the literature. Tomohiro *et al.* [2] worked on palindrome pattern matching

problem which involves computation of palindromic structure in text string that matches with the pattern string by using KMP (Knuth Morris Pratt) - type algorithm. R. Kolpakov *et al.* [1] proposed efficient algorithms for finding long armed gapped palindromes using Lempel Ziv factorization method and length constrained gapped palindromes using suffix array [4, 5, 6, 10, 12, 13, 15] with longest common subword method. S. J. Henstra [3] introduced an algorithm to determine palindrome density in RNA sequences using suffix array and longest common prefix methods.

In this paper, we develop two efficient algorithms for finding long armed and length constrained versions of gapped palindromes in a DNA sequences. Our proposed algorithms use Inverted Suffix Array (ISA) and Longest Common Prefix (LCP) techniques to detect different classes of gapped palindromes. Here, these algorithms perform the computation in $O(n)$ time, where n is the length of input string. Then, we determine palindromic weights in the input. Palindromic weight is represented by the number and size of gapped palindromes found in the biological input string. Also, the proposed algorithm of finding length constrained gapped palindromes is compared with the existing algorithm [1].

The paper is organized as follows. In Sec. 2, we discuss the related concepts. In Sec. 3, we discuss our proposed algorithms. In Sec. 4, we discuss the experimental results and finally we conclude in Sec.5.

II. RELATED CONCEPTS

A. SEARCHING LENGTH CONSTRAINED GAPPED PALINDROMES

The algorithm presented in [1] computes length constrained gapped palindromes by using suffix array and longest common sub-word. It involves the construction of suffix array for the word $w\#w^T$, where w is the sequence and w^T is the reverse complement of w . Then, the arm length of this palindrome was determined by computing the longest common sub-word starting at positions i^- and j^+ . Furthermore, it maintains the pointer from each run to the next run, so that it is able to “jump”, in a constant time, from any item of the whole run. Since new items can be added by the algorithm incrementally, there is need to maintain pointers in a dynamic

manner. The time taken by the algorithm is $O(n+S)$ for a constant size alphabet, where S is the number of output palindromes. The algorithms can be extended to compute different versions of gapped palindromes in biological sequences.

B. PALINDROMES AND GAPPED PALINDROMES IN BIOLOGICAL SEQUENCES

A palindrome in DNA [2, 3, 16] is a sequence of nucleotide bases that reads same as its reverse complement (where A is complementary to T (or U in case of RNA) and C is complementary to G). For example, ACGT is a DNA palindrome such that TGCA is complementary string (where A replace by T, C replace by G, G replace by C and T replace by A) of ACGT, and then reverse the complementary string i.e. ACGT, which is same as original string.

A gapped palindrome in a biological sequence [1] is defined as a sequences vuv^T where v is a non-empty sequence of bases, u (spacer) is a sequence of bases and v^T is the reversed complement of v , the reverse of v with all bases replaced by their potential base pair partners (C replaced by G, G by C, A by T and T by A).

There are two natural classes of gapped palindromes: Long Armed and Length Constrained. A gapped palindrome vuv^T in biological sequence is called long armed if $|u| \leq |v|$, i.e. length of spacer should be less than or equal to the length of palindrome arm where v is a non-empty sequence of bases, u (spacer) is a sequence of bases and v^T is the reversed complement of v .

A gapped palindrome vuv^T in biological sequence is called length constrained if $\text{MinGap} \leq |u| \leq \text{MaxGap}$ and $\text{MinArm} \leq |v|$ such that MinArm , MinGap and MaxGap are some predefined constants; where v is a non-empty sequence of bases, u (spacer) is a sequence of bases and v^T is the reversed complement of v , the reverse of v with all bases replaced by their potential base pair partners (C replaced by G, G by C, A by T and T by A); MinArm , MinGap and MaxGap are predefined constants.

C. SUFFIX ARRAY AND INVERTED SUFFIX ARRAY

Suffix array [3, 10, 12, 13] is defined as the permutation of index numbers giving the starting positions of suffixes of a string in alphabetical order. For example, consider string CGAACG, first construct all suffixes of the string and then arrange all the suffixes in alphabetical order. The suffix array of the string CGAACG is (3; 4; 5; 1; 6; 2) as shown in Table I.

TABLE I. SUFFIX ARRAY AND LCP LENGTH OF CGAACG

INDEX	SUFFIX	LCP	LCP length
3	AACG	A	1
4	ACG	-	0
5	CG	CG	2
1	CGAACG	-	0
6	G	G	1
2	GAACG	-	0

The inverted suffix array is defined as permutation of an index of the suffix array. In the case of an inverted suffix array which contains more than one string, the entries will be in

pairs. The first number denotes the string the suffix belongs to, the second one is the start position of the suffix in this string. Therefore, the inverted suffix array needs to have two dimensions, one for the string and the other for the suffix's start position in the string.

D. LONGEST COMMON PREFIX (LCP) LENGTH

A Longest Common Prefix (LCP) length [3, 11] is the length of the longest common prefix of every two subsequent strings in some array. It is used to improve the performance of algorithms on suffix arrays. Continuing with string CGAACG, the computation of LCP length is shown in Table I. The natural way to compute LCP length is to compare each pair of subsequent suffixes in the suffix array and compare them character by character.

III. PROPOSED ALGORITHMS

A. ALGORITHM TO DETECT LONG ARMED GAPPED PALINDROMES

The algorithm begins with sliding window iterator across the biological input string of length 'n' which slides the window forward by a fixed position for the sequence of length k until entire input string has been processed. It traverses from position 0 to n-k across the input and slides one position forward for new sequence. At some position $j = i$, read each sequence of length k. The DNA sequence GCATTTATGC is fetched by the sliding window iterator at some position. The following steps are followed to determine if the sequence is long armed:

a. Reverse Complement

Reverse Complement is defined as the combination of complementarity of nucleotides (C is complementary to G and A is complementary to T) and then its reversal. The reverse complement of DNA sequence is done in following steps:

- (i) Complement the DNA sequence by replacing A by T, C by G, G by C and T by A which gives above DNA sequence as CGTAAATACG.
- (ii) Reverse the complement which gives GCATAAATGC.

b. Concatenation of # and \$

The DNA sequence is concatenated with the special symbol # and reverse complement with \$ so that new sequence is GCATTTATGC# and GCATAAATGC\$.

c. Construct Inverted Suffix Array

An inverted suffix array is a two dimensional array that contains all suffixes of sequence and its reverse complement, sorted in alphabetical order as shown in Table II. Here, '1' in string ID column represents that suffix belongs to sequence GCATTTATGC# and '2' represents that suffix belongs to reverse complement GCATAAATGC\$. The integer in the 'index' column represents the starting position of the suffix.

TABLE II. INVERTED SUFFIX ARRAY OF GCATTTATGC#
AND GCATAAATGC\$ AND COMPUTATION OF LCP LENGTH

STRING ID	INDEX	SUFFIX	LCP	LCP length
1	11	#	-	0
2	11	\$	-	0
2	5	AAATGC\$	AA	2
2	6	AATGC\$	A	1
2	3	ATAAATGC\$	AT	2
1	7	ATGC#	ATGC	4
2	7	ATGC\$	AT	2
1	3	ATTTATGC#	-	0
1	10	C#	C	1
2	10	C\$	C	1
2	2	CATAAATGC\$	CAT	3
1	2	CATTTATGC#	-	0
1	9	GC#	GC	2
2	9	GC\$	GC	2
2	1	GCATAAATGC\$	GCAT	4
1	1	GCATTTATGC#	-	0
2	4	TAAATGC\$	TA	2
1	6	TATGC#	T	1
1	8	TGC#	TGC	3
2	8	TGC\$	T	1
1	5	TTATGC#	TT	2
1	4	TTTATGC#	-	0

d. Computation of Longest Common Prefix (LCP) Length

The length of LCP is computed by comparing the subsequent suffixes character by character. The number of longest matched prefix characters in two subsequent suffixes is taken as the LCP length. The computation of LCP length is shown in Table II. Then, consider the index numbers of maximum LCP length. Here, index numbers 1 and 7 have maximum LCP length of '4'.

e. Verify certain constraints

According to the definition of long armed gapped palindromes, the length of the spacer should be less than or equal to the length of palindrome arm. Therefore, it is required to verify certain constraints of palindrome arms and spacer between them.

Verify Palindrome Arm Constraints

To verify the constraints of palindrome arm, the following condition should be satisfied: $W(i) = W(j)$ if $i < j$ such that (a) $W(i) = W[i - \text{maximum LCP length} \dots i-1]^T$; (b) $W(j) = W[j \dots j + \text{maximum LCP length} - 1]$ where

$W = W[1 \dots n]$: A DNA sequence, W^T : Reverse Complement of W

$W(i)$: subsequence at position i , i is interpreted as start positions in backward direction respectively.

$W(j)$: subsequence at position j , j is interpreted as start positions in forward direction respectively.

It is required to compute the values of i and j . The value of i and j are 5 and 7 respectively satisfying the above condition ($i < j$). The value of i and j are computed from the index values found in the step c of the algorithm. The value of i is computed from 1, the value of 1 is incremented by one until $W(i)$ could be possibly computed. The value of j is computed from 7, such that $i < j$.

The steps for computation of value of 'i' are as follows:

- As seen from Table II, the index 1 has maximum LCP length of 4.
- For computing i , when we put value of $i=1$ in $W(i) = W[i - \text{maximum LCP length} \dots i-1]^T$, it gives the result as: $W(1) = W[1-4 \dots 1-1]^T$
 $W(1) = W[-3 \dots 0]^T$ which is not possible.
- Then, increment the value of $i=1$ by 1 which becomes 2 i.e. $i=2$.
- Continuing the above steps for computing i , when we put value of $i=5$ in $W(i) = W[i - \text{maximum LCP length} \dots i-1]^T$, it gives the result as: $W(5) = W[5-4 \dots 5-1]^T$
 $W(5) = W[1 \dots 4]^T$ which is possible.

The steps for computation of value of 'j' are as follows:

- As seen from Table II, the index 7 has maximum LCP length of 4.
- For computing j , when we put value of $j=7$ in $W(j) = W[j \dots j + \text{maximum LCP length} - 1]$, it gives the result as: $W(7) = W[7 \dots 7+4-1]$
 $W(7) = W[7 \dots 10]$ which is possible.

Here, the W is the original sequence and W^T is the reverse complement of W such that $W = \text{GCATTTATGC\#}$ and $W^T = \text{GCATAAATGC\$}$. Now, putting the value of $i=5$ in $W(i) = W[i - \text{maximum LCP length} \dots i-1]^T$ gives $W(5) = W[1 \dots 4]^T$. Therefore, the subsequence found in W for $W(5)$ is TACG, started from the backward direction at position 4.

Now, putting the value of $j = 7$ in $W(j) = W[j \dots j + \text{maximum LCP length} - 1]$ gives $W(7) = W[7 \dots 10]$. Therefore, the subsequence found in W for $W(7)$ is ATGC, started from the forward direction at position 7.

Then, the algorithm checks for complementarity (C is complementary to G and A is complementary to T) such that $TACG = ATGC$ satisfying $W(i) = W(j)$.

Verify Spacer Length Constraints

To verify the length constraints of spacer between palindrome arms, the following condition should be satisfied: $j - i \leq \text{Maximum LCP length}$. Hence, we say the DNA sequence GCATTTATGC fetched by sliding window iterator is long armed gapped palindrome.

B. ALGORITHM TO DETECT LENGTH CONSTRAINED GAPPED PALINDROMES

For length constrained gapped palindromes, we define $\text{MinArm} = 3$, $\text{MinGap} = 3$ and $\text{MaxGap} = 5$. The algorithm begins with sliding window iterator across the biological input string of length n which slides the window forward by a fixed position for the sequence of length k until entire input string has been processed. It traverses from position 0 to $n-k$ across the input and slides one position forward for new sequence. At some position $j = i$, read each sequence of length k . The DNA sequence GCATTTATGC is fetched by the sliding window iterator at some position. The following steps are followed to determine if the sequence is length constrained:

- Repeat steps a- c of the algorithm (Sec. III, A) that is used to detect long armed version of gapped palindromes.
- The computation of LCP length is shown in Table II. In this case, consider the index numbers that have LCP length equal to the value of MinArm . Here, index numbers 2 and 8 have LCP length of '3' which is equal to the value of MinArm .

c. Verify Length Constraints

According to the definition of length constrained gapped palindromes, the length of the spacer should not be lesser and greater than the values of MinGap and MaxGap , respectively. Also, the length of palindrome arm should not be lesser than the value of MinArm . Therefore, it is required to test the conditions verifying length constraints of palindrome arms and spacer between palindrome arms.

Verify Palindrome Arm Constraints

To verify the length constraints of palindrome arm, the following condition should be satisfied: $W(i) = W(j)$ if $i < j$ such that (a) $W(i) = W[i - \text{MinArm} \dots i - 1]^T$; (b) $W(j) = W[j \dots j + \text{MinArm} - 1]$ where

$W = W[1 \dots n]$: A DNA sequence, W^T : Reverse Complement of W

$W(i)$: subsequence at position i , i is interpreted as start positions in backward direction respectively.

$W(j)$: subsequence at position j , j is interpreted as start positions in forward direction respectively.

It is required to compute the values of i and j . The value of i and j are 4 and 8 respectively satisfying the above condition ($i < j$). The value of i and j are computed from the index value found in the step of construction of inverted suffix array. The value of i is computed from 2, the value of 2 is incremented by one until $W(i)$ could be possibly computed. The value of j is computed from 8, such that $i < j$.

The steps for computation of value of 'i' are as follows:

- As seen from table II, the index 2 has LCP length of 3.
- For computing i , when we put value of $i = 2$ in $W(i) = W[i - \text{MinArm} \dots i - 1]^T$, it gives the result as: $W(2) = W[2 - 4 \dots 1 - 1]^T$
 $W(2) = W[-2 \dots 0]^T$ which is not possible.
- Then, increment the value of $i = 2$ by 1 which becomes 3 i.e. $i = 3$.
- Continuing the above steps for computing i , when we put value of $i = 4$ in $W(i) = W[i - \text{MinArm} \dots i - 1]^T$, it gives the result as: $W(4) = W[4 - 3 \dots 4 - 1]^T$
 $W(4) = W[1 \dots 3]^T$ which is possible.

The steps for computation of value of 'j' are as follows:

- As seen from table II, the index 8 has LCP length of 3.
- For computing j , when we put value of $j = 8$ in $W(j) = W[j \dots j + \text{MinArm} - 1]$, it gives the result as: $W(8) = W[8 \dots 8 + 3 - 1]$
 $W(8) = W[8 \dots 10]$ which is possible.

Here, the W is the original sequence and W^T is the reverse complement of W such that $W = \text{GCATTTATGC}\#$ and $W^T = \text{GCATAAATGC}\$$. Now, putting the value of $i = 4$ in $W(i) = W[i - \text{MinArm} \dots i - 1]^T$ gives $W(4) = W[1 \dots 3]^T$. Therefore, the subsequence found in W for $W(4)$ is ACG, started from the backward direction at position 3.

Now, putting the value of $j = 8$ in $W(j) = W[j \dots j + \text{MinArm} - 1]$ gives $W(8) = W[8 \dots 10]$. Therefore, the subsequence found in W for $W(8)$ is TGC, started from the forward direction at position 8.

Then, the algorithm checks for complementarity (C is complementary to G and A is complementary to T) such that $ACG = TGC$ satisfying $W(i) = W(j)$.

Verify Spacer Length Constraints

To verify the length constraints of spacer between palindrome arms, the following condition should be satisfied: $\text{MinGap} \leq j - i \leq \text{MaxGap}$. Hence, we say the DNA sequence GCATTTATGC fetched by sliding window iterator is length constrained gapped palindrome.

IV. EXPERIMENTAL RESULTS

The gapped palindrome detection algorithms are tested on existing DNA sequence provided by National Centre for Biotechnology Information (NCBI) website <http://www.>

ncbi.nlm.nih.gov. The proposed algorithm is verified on available datasets such as Bovine testis satellite I (1.715 g/ml) DNA, segment 3, Synthetic vaccine virus- derived cloning vector pMP528HRH with H6 promoter and Human Lymphocyte Antigen (HLA)- DN20 mRNA, 3' flank.

A. DETECTION OF LONG ARMED GAPPED PALINDROMES

Table III provides the results of the long armed gapped palindromes detected by our proposed algorithm.

B. COMPARISON WITH EXISTING ALGORITHM

We compared the execution time of the existing algorithm [1] with our proposed algorithm that computes length constrained gapped palindromes, since the algorithm presented in [1] tackle only this problem. Table IV shows the comparison of computation time of both algorithms.

C. ANALYSIS

Our proposed algorithms were implemented in Turbo C on Windows 8 platform. The programs were tested on Intel® core™ i3 processor with 4.00 GB RAM. Our algorithm begins with the sliding window iterator across the biological input string of length n which slides the window forward by one position for the subsequence of length '10' until the entire input has been processed. Also, our approach computes palindromic weight of the input. From the results, we conclude that the time taken for the computation increases as the number of base pairs in DNA sequence increases.

Here, the technique of inverted suffix array uses two dimensional integer array; the entries in the array will be in pairs. The first denotes the sequence suffix belongs to and second denotes the suffix's starting position. Hence, the consumption of space will be less. Also, the inverted suffix array stores multiple sequences into one array, which eliminates the need of multiple suffix array for different sequences. The overall complexity of the proposed algorithm is $O(n)$.

V. CONCLUSIONS AND FUTURE WORK

We have presented constant time algorithm to detect long armed version of gapped palindromes in DNA sequences by using inverted suffix array and longest common prefix techniques. Also, we have presented an algorithm to detect length constrained version of gapped palindromes in DNA sequences by using inverted suffix array and longest common prefix techniques. Then we determine palindromic weight in the input. Here, we have identified some interesting relation that time of the computation increases as the number of base pair increases. Also, in a long DNA sequences, only certain number of motifs (gapped palindromes) could be found.

This work can be extended for detection of different classes of gapped palindromes in protein sequences. In addition, finding the long armed and length constrained gapped palindromes in biological sequences in which

palindromes have insertions and deletions can also be designed.

REFERENCES

- [1] R. Kolpakov, G. Kucherov, "Searching for gapped palindromes", *Theoretical Computer Science* 410 (51), pp. 5365–5373, 2009.
- [2] Tomohiro I, Shunsuke Inenaga, Masayuki Takeda, "Palindrome Pattern Matching", *Theoretical Computer Science*, pp. 1-9, 2012.
- [3] Sjoerd J. Henstra, "Determining gapped palindrome density in RNA using suffix arrays", *Leiden Institute of Advanced Computer Science Leiden University*, pp. 1-15, 2010.
- [4] M.I. Abouelhoda, S. Kurtz, and E. Ohlebusch, "Enhanced Suffix Array and its Applications".
- [5] M. I. Abouelhoda, Kurtz, S., and E. Ohlebusch, "Replacing Suffix Trees with Enhanced Suffix Arrays", *Journal of Discrete Algorithms* 2, pp. 53–86, 2004.
- [6] M.I. Abouelhoda, S. Kurtz, and E. Ohlebusch, "The Enhanced Suffix Array and its Applications to Genome Analysis", in *Proceedings of the Second Workshop on Algorithms in Bioinformatics, Lecture Notes in Computer Science* 2452, Springer-Verlag, pp. 449–463, 2002.
- [7] D. Gusfield, "Algorithms on Strings, Trees, and Sequences", *Cambridge University Press*, New York, 1997.
- [8] D. Gusfield and J. Stoye, "Linear Time Algorithms for Finding and Representing all the Tandem Repeats in a String", *Report CSE-98-4, Computer Science Division, University of California, Davis*, 1998.
- [9] M. H'ohl, S. Kurtz and E. Ohlebusch, "Efficient Multiple Genome Alignment", *Bioinformatics* 18, Suppl. 1, pp. 312–320, 2002.
- [10] J. Kärkkäinen and P. Sanders, "Simple Linear Work Suffix Array Construction" in *Proc. International Colloquium on Automata, Languages and Programming, Lecture Notes in Computer Science* 2719, Springer Verlag, pp. 943–955, 2003.
- [11] T. Kasai, G. Lee, H. Arimura, S. Arikawa, and K. Park, "Linear-Time Longest Common Prefix Computation in Suffix Arrays and its Applications", in *Proceedings of the 12th Annual Symposium on Combinatorial Pattern Matching*, *Lecture Notes in Computer Science* 2089, Springer-Verlag, pp. 181–192, 2001.
- [12] D. K. Kim, J.S. Sim, H. Park and K. Park, "Linear-Time Construction of Suffix Arrays", in *Proceedings of the 14th Annual Symposium on Combinatorial Pattern Matching (CPM)*, vol. 2676 of LNCS, Springer-Verlag, pp. 186–199, 2003.
- [13] P. Ko and S. Aluru, "Space Efficient Linear Time Construction of Suffix Arrays", in *Proceedings of the 14th Annual Symposium on Combinatorial Pattern Matching (CPM)*, vol. 2676 of LNCS, Springer-Verlag, pp. 200–210, 2003.
- [14] R. Kolpakov, and G. Kucherov, "Finding Maximal Repetitions in a Word in Linear Time" in *Symposium on Foundations of Computer Science, IEEE Computer Society*, pp. 596–604, 1999.
- [15] S. Kurtz, "Reducing the Space Requirement of Suffix Trees", *Software—Practice and Experience* 29, pp. 1149–1171, 1999.
- [16] Ravi Gupta, Ankush Mittal, Sumit Gupta, "An efficient algorithm to detect palindromes in DNA sequences using periodicity transform", *Signal Processing* 86, pp. 2067–2073, 2006.
- [17] Alexandre H.L. Porto, Valmir C. Barbosa, "Finding approximate palindromes in strings", *Pattern Recognition* 35, pp. 2581–2591, 2002.

- [18] M.-C. Anisiu, V. Anisiu, Z. Kása, “Total palindrome complexity of finite words”, Discrete Mathematics 310 (1), pp. 109–114, 2010.
- [19] A. Glen, J. Justin, S. Widmer, L.Q. Zamboni, “Palindromic richness”, European Journal of Combinatorics 30 (2), pp.510–531, 2009.
- [20] P.H. Hsu, K.Y.Chen and K.-M. Chao, “Finding all approximate gapped palindromes”, in: Proc. ISAAC, LNCS, vol. 5878, pp. 1084–1093, 2009.

Table III. LONG ARMED GAPPED PALINDROMES DETECTED IN STANDARD DATASETS

DATASET	Total base pairs in dataset	Palindrome Weight		Length of Palindrome arm	Spacer Length	Long Armed Gapped Palindromes	Time (in ms)
		Number of Palindromes	Size of Palindrome				
Bovine testis satellite I (1.715 g/ml) DNA, segment 3	131 bp	1	10	4	2	CGCTGTAGCG	0.769231
Synthetic vaccine virus-derived cloning vector pMP528HRH with H6 promoter	141 bp	0	-	-	-	-	1.134829
Human Lymphocyte Antigen (HLA-DN20 mRNA, 3' flank	230 bp	1	10	4	2	ATTTAAAAAT	1.3186818

TABLE IV. COMPARISON OF PROPOSED ALGORITHM AND EXISTING ALGORITHM

DATASET	Total base pairs in Dataset	Palindrome weight		Palindrome Arm Length	Spacer Length	Length Constrained Gapped Palindromes	Time (in ms) by existing algorithm [1]	Time (in ms) by proposed algorithm
		Number of Palindromes	Size of Palindrome					
Bovine testis satellite I (1.715 g/ml) DNA, segment 3	131 bp	1	10	3	4	GCGTTTACG C	1.373626	1.043956
Synthetic vaccine virus-derived cloning vector pMP528HRH with H6 promoter	141 bp	1	10	3	4	TCGTACTCG A	1.482974	1.098901
Human Lymphocyte Antigen (HLA-DN20 mRNA, 3' flank	230 bp	4	10 10 10	3 3 3	4 4 4	TCTACAAAG A ATTTAAAAA T CCAGGCATG G GGCTGGGGC C	1.538462	1.208791